

GitHub Copilot Chat History

Project: Word Scramble Arena (Pygame)

Repository: github.com/sahanagolabanvi34-debug/62_scramble

Source: VS Code Copilot Chat Session export provided by the student. User prompts and Copilot statements have been extracted into a readable transcript. Technical metadata, internal reasoning records, file snapshots and tool-call payloads are omitted.

Scope of this export: Hint button, countdown timer, and interactive letter tiles. The original answer-validation fix was completed separately and is not represented as a Copilot conversation in this export.

Feature 1 - Hint Button

Student prompt

Add a Hint button to my existing Pygame Word Scramble game. Each click should reveal one more letter of the correct word in order. Deduct 1 score point per hint, but never let the score go below zero. Reset the hints when a new word appears. Keep all existing game functionality working. Modify only the necessary files.

Copilot conversation (readable transcript)

I'll trace the round lifecycle and rendering in the existing engine, then make the smallest focused change and run a quick syntax or behavior check. I'll also check the repo's local instructions before editing.

The engine owns both round changes and mouse handling, so the feature fits entirely in `game_engine.py`. I'll track how many leading letters are exposed, reset that count in `game_engine.py`, and only charge for a click that reveals a new letter; this keeps repeated clicks after the whole word is shown from charging for a no-op.

The focused runtime check passes, including the score floor and round reset. I spotted one tiny presentation issue in the masked string: the first revealed letter runs directly into the next placeholder, so I'm spacing the displayed characters consistently and rerunning the same check.

Added a Hint button and ordered letter reveals in `game/game_engine.py`. Each new reveal costs 1 point, the score stays at zero or above, and hints reset when a new word appears. Existing submit

behavior is unchanged. Verified with a focused runtime check covering hint progression, score clamping, completed reveals, and round reset.

Verification recorded in the export

Test output: Hint behavior checks passed

Feature 2 - 30-Second Countdown Timer

Student prompt

Add a 30-second countdown timer to my existing Pygame Word Scramble game. Display the remaining seconds clearly on screen. Start the timer whenever a new word appears. When the timer reaches zero, automatically move to a new scrambled word and reset the timer. Reset the timer after a correct answer too. Use `pygame.time.get_ticks()` without blocking the game loop. Preserve the existing Hint button, score system, and Submit functionality. Modify only necessary files and test the timer.

Copilot conversation (readable transcript)

I'll use the existing round transition as the timer's reset point and `game_engine.py` for the non-blocking expiration check. First I'm confirming the current engine structure; then I'll add the countdown display and exercise both timeout and correct-answer resets with a controlled clock.

The round lifecycle confirms the simplest control path: `game_engine.py` already runs at startup and after correct answers. I'll anchor the 30,000 ms start time there, advance rounds from the existing per-frame `game_engine.py`, and derive the displayed seconds from `game_engine.py` so nothing blocks the loop.

The first test invocation returned to the PowerShell prompt without running the test body, so it didn't give us a result. I'm rerunning the same controlled-clock checks as a single Python command to verify expiration and both round-reset paths.

Added a visible 30-second countdown to `game/game_engine.py`. It uses `game_engine.py` and checks expiration in `game_engine.py`, so the loop stays non-blocking. Timeout and correct answers both start a fresh timer; the Hint button and score behavior remain intact. Verified countdown rendering, the 30-second timeout, automatic round change, and correct-answer reset using a mocked clock.

Verification recorded in the export

Test output: Timer timeout, countdown rendering, and correct-answer reset checks passed

Feature 3 - Interactive Letter Tiles

Student prompt

Add interactive clickable letter tiles to my existing Pygame Word Scramble game. Display each scrambled letter inside a separate clickable tile. When a player clicks a tile, append its letter to the answer input and mark the tile as selected so it cannot be used twice. Provide a way to undo or clear selected letters. Correctly handle repeated letters, such as the two Zs in PUZZLE. Keep keyboard input working. Reset tile selections for every new word. Preserve the Hint button, 30-second countdown timer, score system, and Submit functionality. Modify only necessary files and test the feature.

Copilot conversation (readable transcript)

I'll keep this inside the game engine: tile hitboxes can follow the scrambled word, and per-character source tracking will let Undo release the correct tile even when keyboard input is mixed in. I'm checking the current input and click flow once, then I'll add the tiles plus Undo/Clear controls and test repeated letters and keyboard coexistence.

The current flow supports this without touching `text_box.py` : I can observe its normal keyboard edits and keep tile/source state synchronized in the engine. I'm adding index-based selection (so repeated letters remain distinct), resetting it on wrong guesses and new rounds, and placing Undo/Clear below the answer row.

The tile state now tracks positions rather than letter values, so both 'Z' tiles in 'PUZZLE' are independently selectable. I've also made keyboard additions/backspaces update that state, which lets Undo and Clear work predictably alongside typed letters; next I'm exercising those interactions and a new-round reset.

The first interaction check exposed a real keyboard-focus issue: `text_box.py` deactivates itself on tile clicks, so subsequent typing would be ignored. I'm keeping the input focused when a game control or tile is clicked, while retaining the normal click-to-focus behavior for the answer box; then I'll rerun the mixed-input test.

The mixed interaction check now passes, including separate 'Z' selection, duplicate-click blocking, Undo, Clear, typing after tile clicks, and resetting selections for a new round. I'm doing one final integration check on the preserved Hint, Submit/score, and countdown behavior.

Added clickable letter tiles, with each position tracked separately so repeated letters such as the two 'Z's in 'PUZZLE' work independently. Added Undo and Clear controls, kept keyboard typing active

after tile clicks, and reset tile selections for each new word. Changes are limited to game/game_engine.py . Verified repeated-letter selection, duplicate-click blocking, Undo/Clear, mixed keyboard input, round resets, Hint deductions, Submit scoring, timer timeout/reset, and rendering.

Verification recorded in the export

Test output: Tile selection, duplicate letters, undo, clear, keyboard sync, round reset, and rendering checks passed

Test output: Hint, submit score, timer reset/timeout, and full render checks passed

End of extracted Copilot conversation